# DEZSYS_GK71_FLOODMONITOR_REST

Akkus Muhammed

# Inhalt

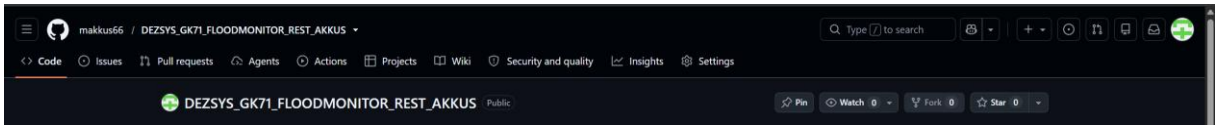
1. Projekt einrichten ..... 4
1.3 In IntelliJ öffnen ..... 4
1.4 Jackson XML hinzufügen ..... 4
1.5 Mit Gradle starten ..... 5
1.6 Testen ..... 5
1.7 .gitignore + Commit ..... 5
2. Messwerte simulieren ..... 6
2a Model ..... 6
2b Repository ..... 6
2c Simulator ..... 6
Simulationsregeln ..... 6
Punkt 3: Warnstufe berechnen ..... 7
3.1 Neue Klasse ..... 7
3.2 SimulationService.java anpassen ..... 7
a) Feld + Konstruktor ersetzen: ..... 7
b) In simulate() die Zeilen ab Measurement next = ... bis zum log.info(...) ersetzen: ..... 8
3.3 Testen ..... 8
3.4 Commit ..... 8
Punkt 4: REST-Schnittstelle ..... 8
4.1 Neues Paket exception ..... 8
4.2 In model zwei neue Klassen ..... 9
Alert.java ..... 9
Statistics.java ..... 9
4.3 In service zwei neue Klassen ..... 10
StationService.java ..... 10
MeasurementService.java ..... 10
4.4 Controller ..... 11
StationController.java komplett ersetzen ..... 11
Neue Klasse controller/AlertController.java ..... 11
4.5 Testen ..... 11
Punkt 5: JSON und XML (Content Negotiation) ..... 12
5.1 Ohne Accept-Header → JSON ..... 12
5.2 JSON explizit ..... 12
5.3 XML ..... 12
5.4 Nicht unterstütztes Format → 406 ..... 13

6. HTTP-Statuscodes und Fehlerbehandlung ..... 13
6.1 Im Paket exception drei neue Klassen ..... 13
6.2 Validierung ..... 13
6.3 Testen ..... 13
6.4 Commit ..... 14
7. Webclient ..... 14
7.1 Ordner anlegen ..... 14
7.2 static/index.html ..... 14
7.3 static/css/style.css ..... 14
7.4 static/js/app.js ..... 14
7.5 Testen ..... 14
Gesamtansicht (JSON) ..... 15
XML-Modus ..... 15
Filter nach Warnstufe und Limit ..... 16
Filter nach Zeitraum ..... 16
Fehlermeldung bei ungültigem Zeitraum ..... 16
Automatische Aktualisierung ..... 17
8. Architektur ..... 17
Punkt 9: Automatisierte Tests ..... 18
9.3 FloodMonitorApiTest.java ..... 18
9.4 WarningLevelServiceTest.java ..... 18
9.5 Tests ausführen ..... 18
Punkt 10: API-Dokumentation mit OpenAPI ..... 19
10.2 Neues Paket config → OpenApiConfig.java ..... 19
10.3 StationController.java komplett ersetzen ..... 19
10.4 AlertController.java komplett ersetzen ..... 19
10.5 Testen ..... 19
Endpunkt mit Parametern und Statuscodes ..... 20
Direkt in Swagger testen (XML) ..... 21
Schemas ..... 22
12. Fragestellungen ..... 23
1. Was sind die Hauptmerkmale von JSON? ..... 23
2. Was sind die Hauptmerkmale von XML? ..... 23
3. Wie unterscheidet die Applikation, ob JSON oder XML zurückgegeben wird? ..... 24
4. Was zeichnet ein REST-Interface aus und warum wird es so oft verwendet? ..... 24
5. Unterschied zwischen GET, POST, PUT und PATCH? ..... 25

6. Welche Stellen im Code muss ich für eine neue Methode / einen neuen Pfad ändern? .....25
7. Kann man OpenAPI und Swagger vergleichen? .....25
8. Wo liegt im Code-Snippet der Fehler? .....26
AI-Prompts.....27

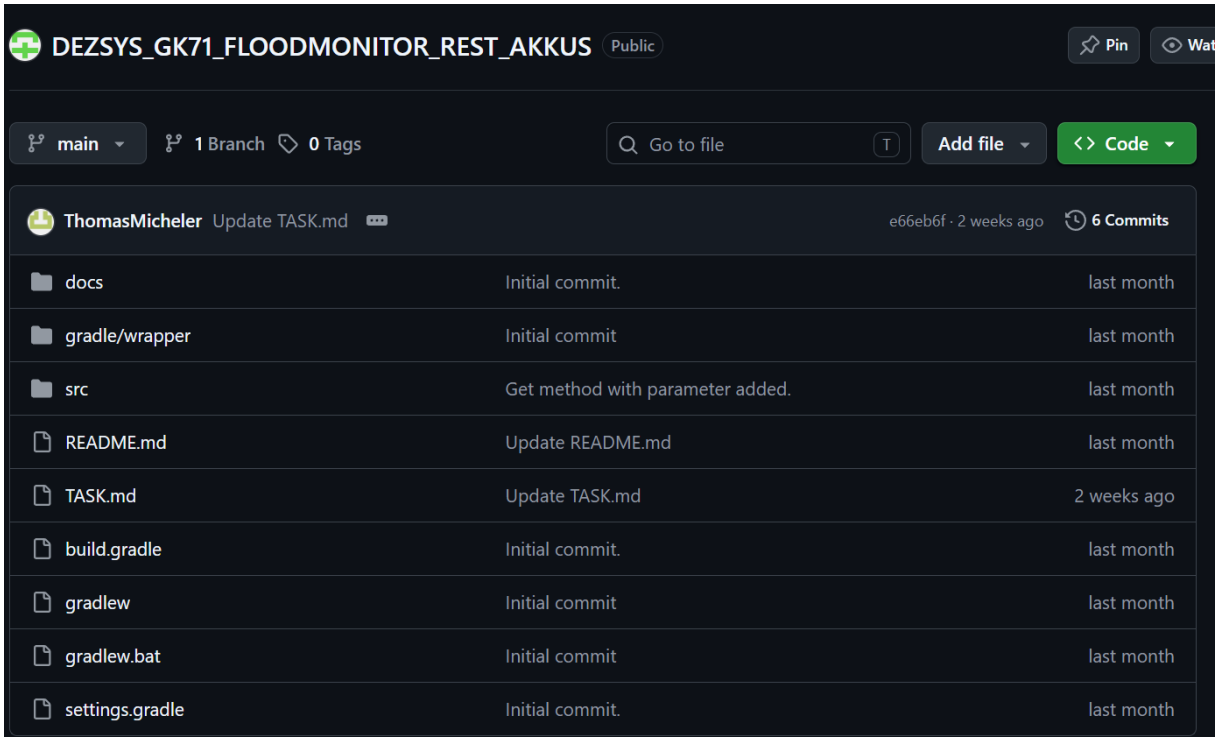
# 1. Projekt einrichten

Zuerst eine Repo erstellen :




Ich habe das Starter-Projekt geklont und in mein eigenes Repository gepusht.

git clone
https://github.com/ThomasMicheler/DEZSYS_GK71_FLOODMONITOR_REST_INCOMPLETE.git
DEZSYS_GK71_FLOODMONITOR_REST_AKKUS cd
DEZSYS_GK71_FLOODMONITOR_REST_AKKUS git remote set-url origin
https://github.com/makkus66/DEZSYS_GK71_FLOODMONITOR_REST_AKKUS.git git push -u
origin main




## 1.3 In IntelliJ öffnen

**File → Open** → Ordner DEZSYS_GK71_FLOODMONITOR_REST_AKKUS

## 1.4 Jackson XML hinzufügen

Ich habe in der build.gradle die Abhängigkeit Jackson XML hinzugefügt. Spring Web war bereits vorhanden.

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'com.fasterxml.jackson.dataformat:jackson-dataformat-xml'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

## 1.5 Mit Gradle starten

Ich habe das Projekt mit Gradle über bootRun gestartet. Der Server läuft auf Port 8080.




## 1.6 Testen

Ich habe im Browser geprüft, dass der Endpunkt /api/v1/stations Daten liefert.

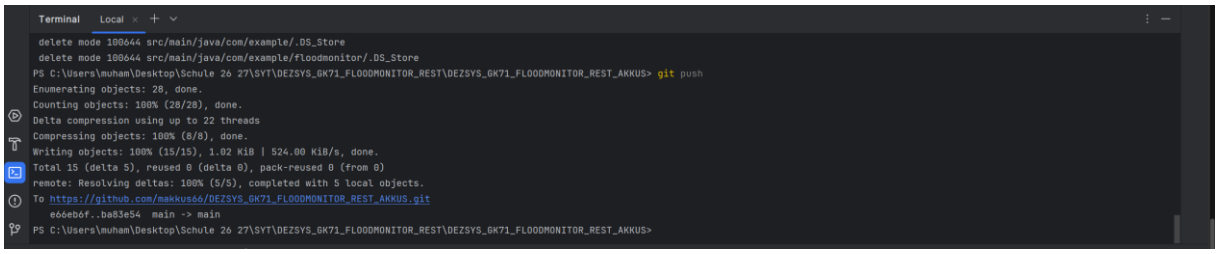
http://localhost:8080/api/v1/stations

This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
▼<List>
  ▼<item>
     <id>1</id>
     <stationName>Station Vienna</stationName>
     <timestamp>2026-10-29T19:11:26.5711423</timestamp>
     <waterLevel>3.76</waterLevel>
     <waterTemperature>16.0</waterTemperature>
     <unit>m</unit>
   </item>
  ▼<item>
     <id>2</id>
     <stationName>Station Linz</stationName>
     <timestamp>2026-10-29T19:11:26.5711423</timestamp>
     <waterLevel>4.73</waterLevel>
     <waterTemperature>11.5</waterTemperature>
     <unit>m</unit>
   </item>
  ▼<item>
     <id>3</id>
     <stationName>Station Salzburg</stationName>
     <timestamp>2026-10-29T19:11:26.5711423</timestamp>
     <waterLevel>4.79</waterLevel>
     <waterTemperature>19.3</waterTemperature>
     <unit>m</unit>
   </item>
  ▼<item>
     <id>4</id>
     <stationName>Station Graz</stationName>
     <timestamp>2026-10-29T19:11:26.5711423</timestamp>
     <waterLevel>2.59</waterLevel>
     <waterTemperature>7.8</waterTemperature>
     <unit>m</unit>
   </item>
  ▼<item>
     <id>5</id>
     <stationName>Station Innsbruck</stationName>
     <timestamp>2026-10-29T19:11:26.5711423</timestamp>
     <waterLevel>3.1</waterLevel>
     <waterTemperature>15.8</waterTemperature>
     <unit>m</unit>
   </item>
 </List>
```

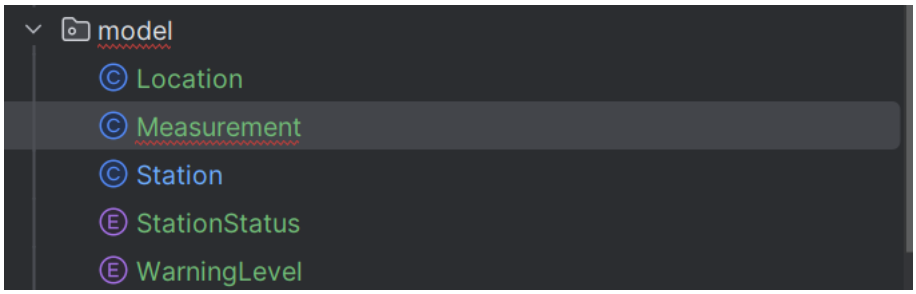

## 1.7 .gitignore + Commit

Ich habe die Änderungen committet und auf GitHub gepusht.




# 2. Messwerte simulieren

## 2a Model



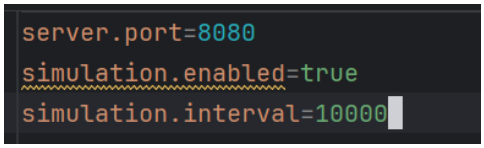

Ich habe das Datenmodell mit den Klassen Station, Location, Measurement und den Enums WarningLevel und StationStatus erstellt.

## 2b Repository

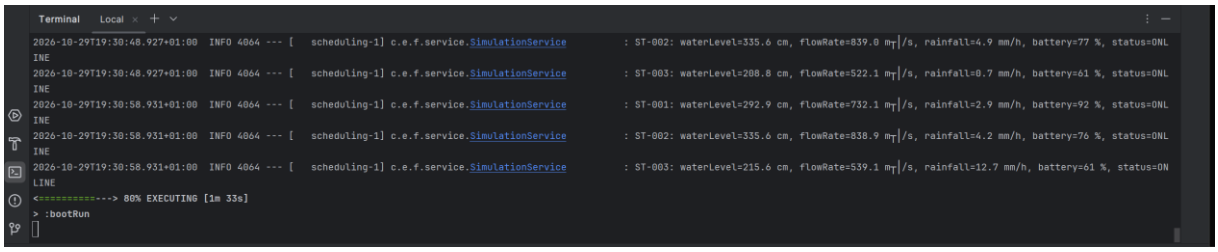
StationRepository und MeasurementRepository erstellt

## 2c Simulator

Bei SimulationService den ganzen code ändern . Hier die FloodMonitorApplication über der Klasse @EnableScheduling ergänzen. application.properties ersetzen Das Simulationsintervall kann ich in der application.properties über simulation.interval einstellen.




StationController.java ersetzen . Dann mit .\gradlew.bat bootRun einfach testen




Dann commiten am ende

### Simulationsregeln

Ich habe folgende Regeln für die Simulation festgelegt:

- Der Niederschlag nimmt pro Messung auf 60 % ab, dazu kommen 0 bis 3 mm/h. Mit 10 % Wahrscheinlichkeit gibt es Starkregen (+10 bis 30 mm/h).
- Der Wasserstand steigt um 0,8 cm pro mm/h Regen und geht pro Messung um 5 % der Abweichung zum Normalwert zurück. Er wird nie negativ.
- Die Durchflussmenge ist 2,5 × Wasserstand, also steigt sie mit dem Pegel.
- Der Akku verliert mit 30 % Wahrscheinlichkeit 1 % pro Messung. Unter 10 % geht die Station in MAINTENANCE und lädt um 20 % pro Messung, bei 100 % ist sie wieder ONLINE.
- Deaktivierte oder OFFLINE-Stationen bekommen keine Messwerte.

# Punkt 3: Warnstufe berechnen

## 3.1 Neue Klasse

service/WarningLevelService.java

## 3.2 SimulationService.java anpassen

Ich habe die Warnstufenberechnung in einen eigenen Service ausgelagert. Die Warnstufe wird nur am Server berechnet.

### a) Feld + Konstruktor ersetzen:

```
@Service  akkusm66 +1 *
@ConditionalOnProperty(name = "simulation.enabled", havingValue = "true", matchIfMissing = true)
public class SimulationService {
    private static final Logger log = LoggerFactory.getLogger(SimulationService.class);  1 usage
    private final StationRepository stationRepository;  2 usages
    private final MeasurementRepository measurementRepository;  3 usages
    private final WarningLevelService warningLevelService;  2 usages
    private final Random random = new Random();  7 usages

    public SimulationService(StationRepository stationRepository,  akkusm66 *
                             MeasurementRepository measurementRepository,
                             WarningLevelService warningLevelService) {
        this.stationRepository = stationRepository;
        this.measurementRepository = measurementRepository;
        this.warningLevelService = warningLevelService;
    }
```

### b) In simulate() die Zeilen ab Measurement next = ... bis zum log.info(...) ersetzen:

```
@Scheduled(fixedRateString = "${simulation.interval:10000}")  akkusm66 *
public void simulate() {
    for (Station station : stationRepository.findAll()) {
        if (!station.isActive() || station.getStatus() == StationStatus.OFFLINE) {
            continue;
        }
        Measurement previous = measurementRepository.findLatest(station.getId()).orElse( other: null);
        Measurement next = nextMeasurement(station, previous);
        next.setWarningLevel(warningLevelService.calculate(station, next));
        measurementRepository.save(next);
        log.info("{}: waterLevel={} cm, flowRate={} m3/s, rainfall={} mm/h, battery={} %, status={}, warningLevel={}",
                next.getStationId(), next.getWaterLevel(), next.getFlowRate(),
                next.getRainfall(), next.getBatteryLevel(), next.getStatus(), next.getWarningLevel());
    }
}
```

## 3.3 Testen

```
Terminal  Local
2026-10-29T19:43:10.717+01:00  INFO 30296 --- [   scheduling-1] c.e.f.service.SimulationService         : ST-002: waterLevel=302.1 cm, flowRate=755.2 m3/s, rainfall=1.9 mm/h, battery=83 %, status=ONLI
NE, warningLevel=NORMAL
2026-10-29T19:43:10.717+01:00  INFO 30296 --- [   scheduling-1] c.e.f.service.SimulationService         : ST-003: waterLevel=202.0 cm, flowRate=504.9 m3/s, rainfall=2.9 mm/h, battery=78 %, status=ONLI
NE, warningLevel=NORMAL
2026-10-29T19:43:20.717+01:00  INFO 30296 --- [   scheduling-1] c.e.f.service.SimulationService         : ST-001: waterLevel=282.0 cm, flowRate=705.1 m3/s, rainfall=2.0 mm/h, battery=82 %, status=ONLI
NE, warningLevel=NORMAL
2026-10-29T19:43:20.717+01:00  INFO 30296 --- [   scheduling-1] c.e.f.service.SimulationService         : ST-002: waterLevel=305.4 cm, flowRate=763.5 m3/s, rainfall=1.6 mm/h, battery=83 %, status=ONLI
NE, warningLevel=NORMAL
2026-10-29T19:43:20.717+01:00  INFO 30296 --- [   scheduling-1] c.e.f.service.SimulationService         : ST-003: waterLevel=202.9 cm, flowRate=507.4 m3/s, rainfall=3.7 mm/h, battery=78 %, status=ONLI
NE, warningLevel=NORMAL
<==========---> 80% EXECUTING [13s]
> :bootRun
```

## 3.4 Commit

Ich berechne die Warnstufe nur anhand des Wasserstands:

| Bedingung | Warnstufe |
|---|---|
| keine Messung, Messung älter als 5 Minuten oder Wasserstand ungültig | UNKNOWN |
| Wasserstand ≥ kritischer Wert | CRITICAL |
| Wasserstand ≥ Warnwert | WARNING |
| sonst | NORMAL |

Niederschlag und Anstiegsgeschwindigkeit habe ich nicht einbezogen, weil der Niederschlag in meiner Simulation schon direkt den Wasserstand erhöht.

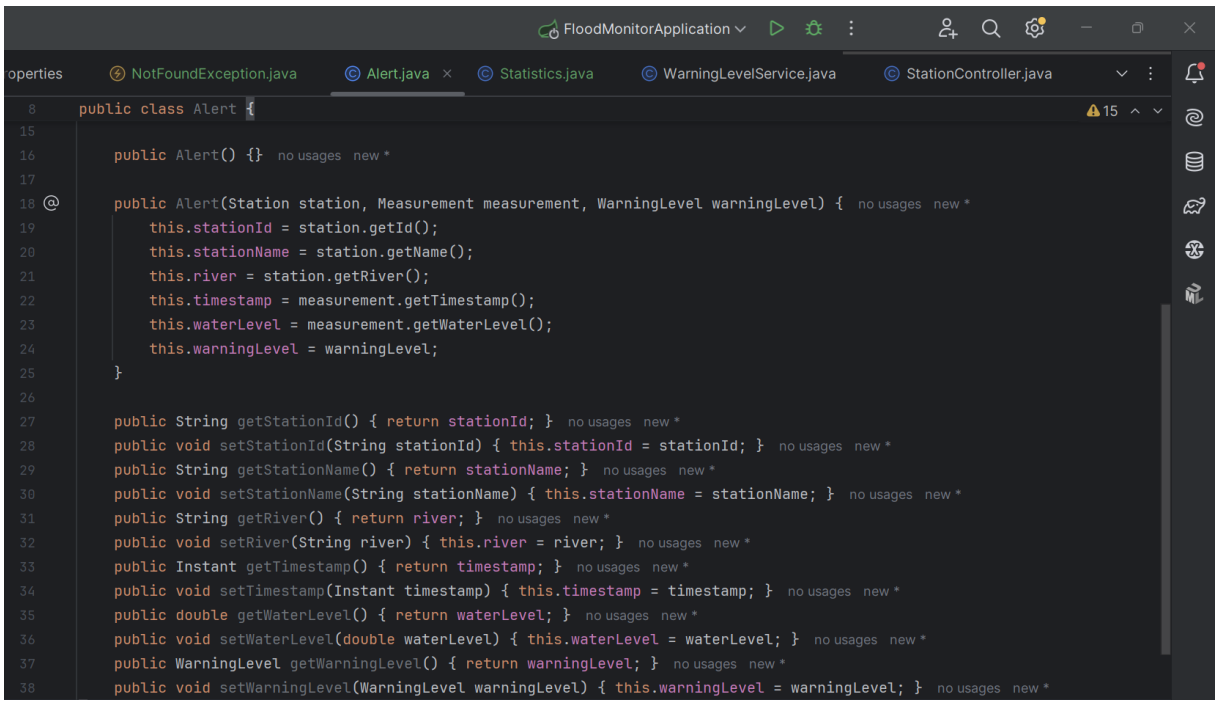
# Punkt 4: REST-Schnittstelle

## 4.1 Neues Paket exception

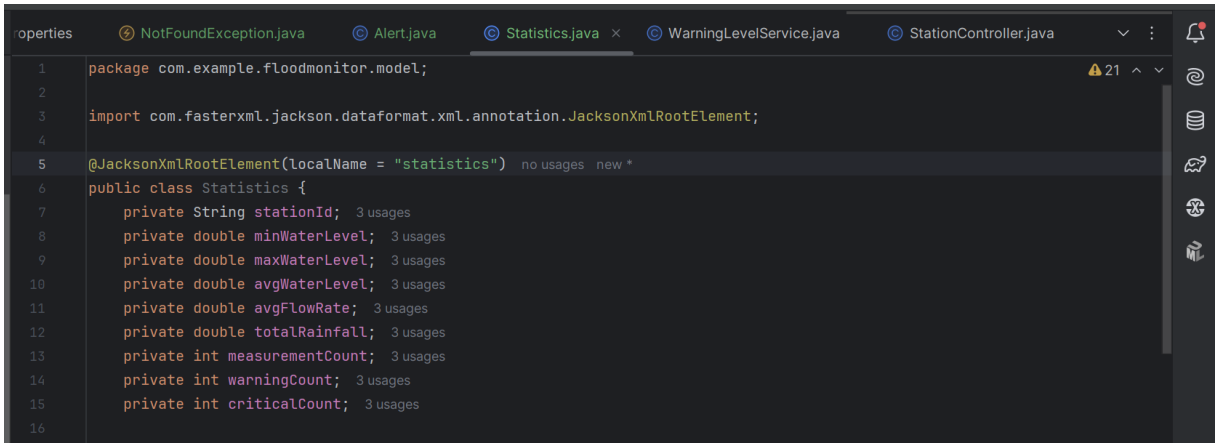
```
tRepository.java  SimulationService.java  FloodMonitorApplication.java  application.properties  NotFoundException.java
1  package com.example.floodmonitor.exception;
2
3  public class NotFoundException extends RuntimeException {  no usages  new *
4      public NotFoundException(String message) {  no usages  new *
5          super(message);
6      }
7  }
```

## 4.2 In model zwei neue Klassen

### Alert.java



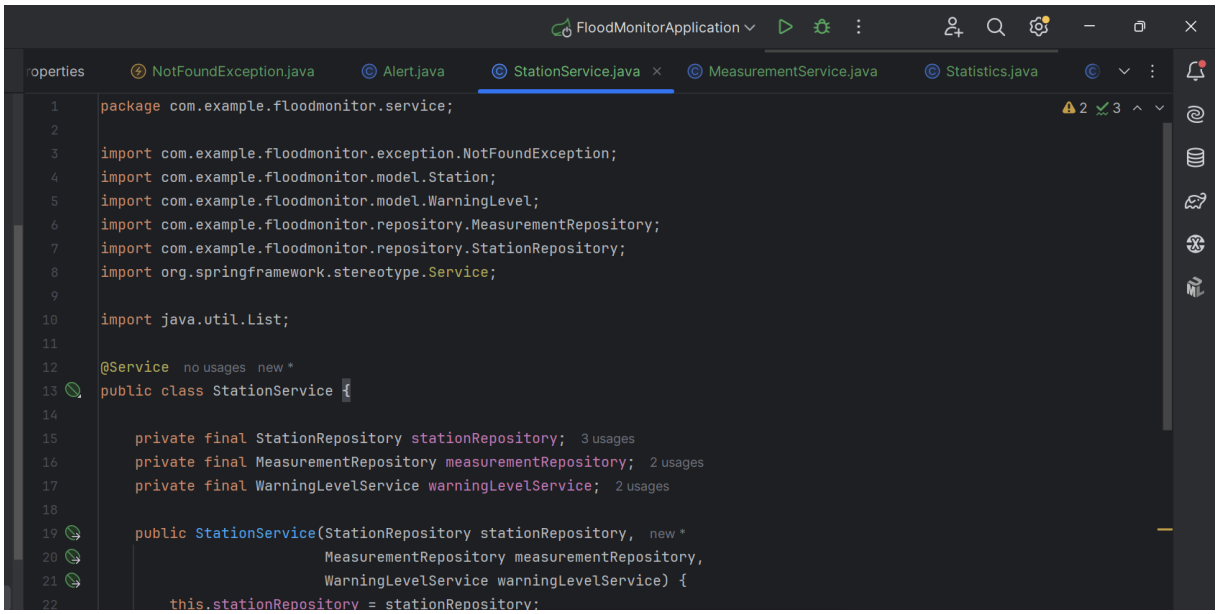

### Statistics.java



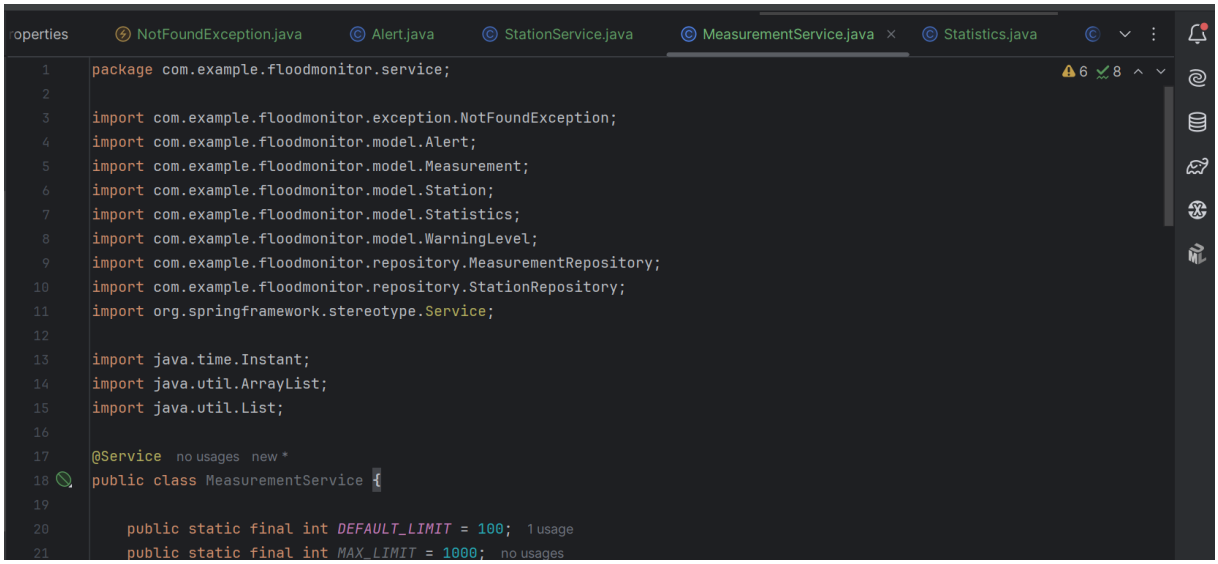

Dies geht noch weiter .

## 4.3 In service zwei neue Klassen

### StationService.java



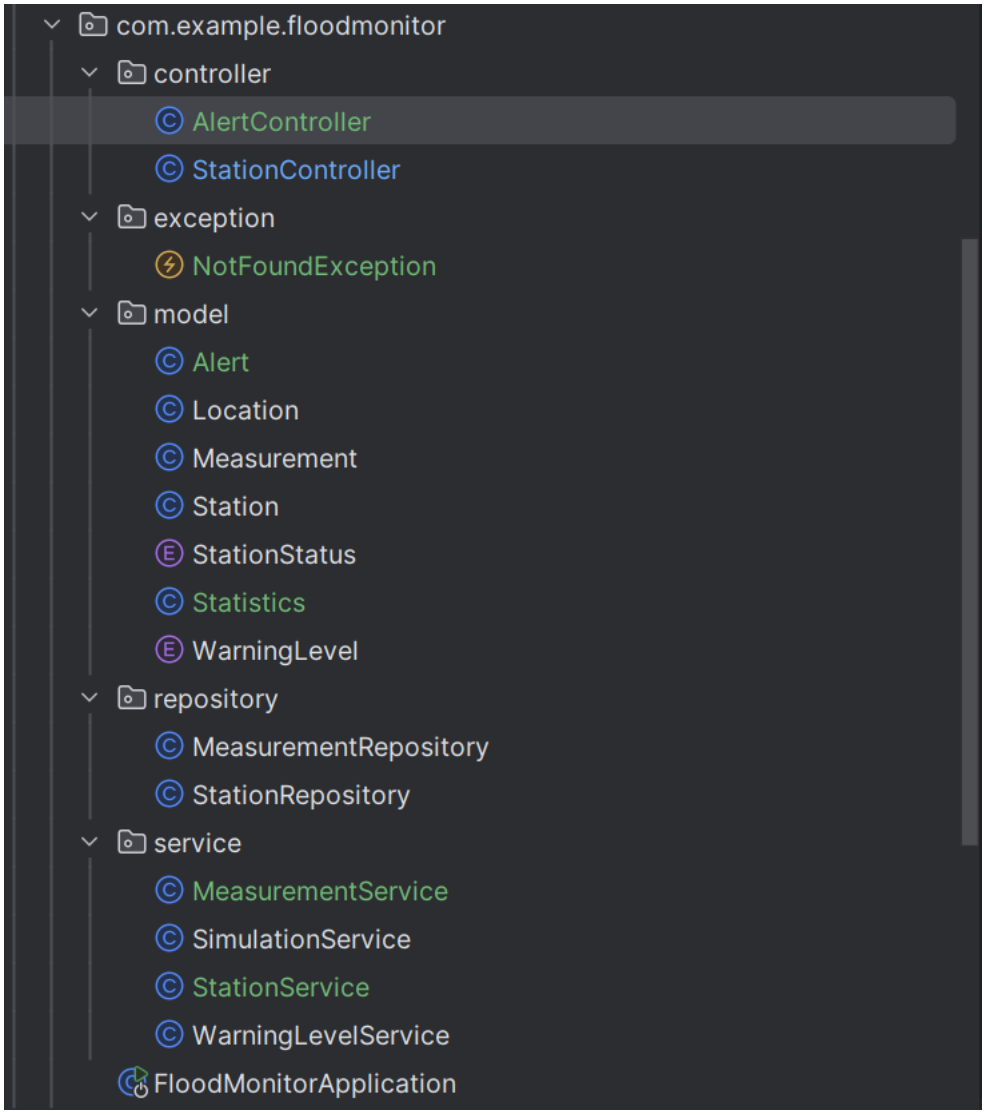

### MeasurementService.java



## 4.4 Controller

### StationController.java komplett ersetzen

### Neue Klasse controller/AlertController.java




## 4.5 Testen

Dann diese sachen aufmachen und diese sachen sollten kommen :

| URL | Erwartung |
|---|---|
| localhost:8080/api/v1/stations | 4 Stationen |
| localhost:8080/api/v1/stations?river=Donau | nur ST-001 und ST-002 |
| localhost:8080/api/v1/stations?isActive=false | nur ST-004 |
| localhost:8080/api/v1/stations/ST-001 | eine Station |

| URL | Erwartung |
|---|---|
| localhost:8080/api/v1/stations/ST-001/measurements/latest | letzter Messwert |
| localhost:8080/api/v1/stations/ST-001/measurements?limit=3 | die 3 neuesten Messwerte |
| localhost:8080/api/v1/stations/ST-001/statistics | Statistik |
| localhost:8080/api/v1/alerts | meist [], weil gerade keine Warnung aktiv ist |

Den Pfad für die Statistik habe ich selbst festgelegt: GET /api/v1/stations/{stationId}/statistics mit optionalem from und to. warningCount zählt WARNING-Messungen, criticalCount die CRITICAL-Messungen.

# Punkt 5: JSON und XML (Content Negotiation)

## 5.1 Ohne Accept-Header → JSON

Server starten neuen terminal öffnen und dies eingeben : curl.exe -i http://localhost:8080/api/v1/stations/ST-001/measurements/latest

Ohne Accept-Header liefert die Schnittstelle standardmäßig JSON (Content-Type: application/json)




## 5.2 JSON explizit

Mit Accept: application/json bekomme ich die Messung als JSON.




## 5.3 XML

Mit Accept: application/xml bekomme ich dieselbe Messung als XML. JSON und XML enthalten dieselben Informationen.



## 5.4 Nicht unterstütztes Format → 406

Bei einem nicht unterstützten Format antwortet die Schnittstelle mit 406 Not Acceptable.

```
PS C:\Users\muham\Desktop\Schule 26 27\SYT\DEZSYS_GK71_FLOODMONITOR_REST\DEZSYS_GK71_FLOODMONITOR_REST_AKKUS> curl.exe -i -H "Accept: text/plain" http://localhost:8080/api/v1/stations/ST-001/measurements/latest
HTTP/1.1 406
Accept: application/json, application/*+json, application/xml;charset=UTF-8, text/xml;charset=UTF-8, application/*+xml;charset=UTF-8
Content-Length: 0
Date: Thu, 29 Oct 2026 19:04:31 GMT

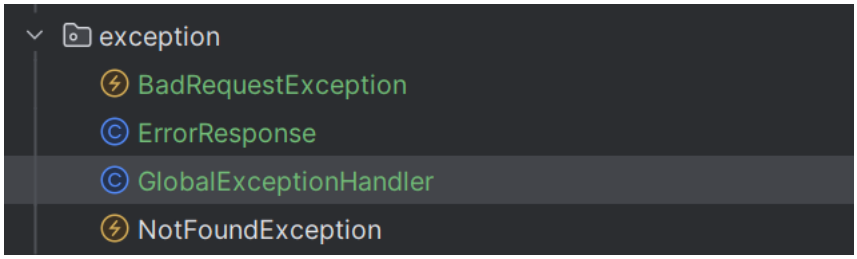
PS C:\Users\muham\Desktop\Schule 26 27\SYT\DEZSYS_GK71_FLOODMONITOR_REST\DEZSYS_GK71_FLOODMONITOR_REST_AKKUS>
```

Der Client wählt das Format über den Accept-Header. Spring Boot vergleicht diesen Header mit den verfügbaren HttpMessageConvertern: Für application/json wird der Jackson-JSON-Converter verwendet, für application/xml der Jackson-XML-Converter (durch die Abhängigkeit jackson-dataformat-xml). Ohne Accept-Header wird JSON geliefert, weil Spring Boot den XML-Converter ans Ende der Liste stellt. Passt kein Converter, antwortet Spring mit 406 Not Acceptable.

# 6. HTTP-Statuscodes und Fehlerbehandlung

## 6.1 Im Paket exception drei neue Klassen

GlobalExceptionHandler : Ich habe eine zentrale Fehlerbehandlung mit @RestControllerAdvice erstellt. Alle Fehler werden einheitlich mit timestamp, status, error, message und path ausgegeben, ohne Stacktrace.




## 6.2 Validierung

MeasurementService.java : Neu sind die Prüfungen für den Zeitraum und das limit

## 6.3 Testen

Server im ersten Terminal starten dann im zweiten Terminal die befehle Testen ein Beispiel :

```
PS C:\Users\muham\Desktop\Schule 26 27\SYT\DEZSYS_GK71_FLOODMONITOR_REST\DEZSYS_GK71_FLOODMONITOR_REST_AKKUS> curl.exe -i http://localhost:8080/api/v1/stations/ST-999
HTTP/1.1 404
Content-Type: application/json
Transfer-Encoding: chunked
Date: Thu, 29 Oct 2026 19:10:40 GMT
```

| Test | Befehl | Erwartet |
|---|---|---|
| Unbekannte Station | curl.exe -i http://localhost:8080/api/v1/stations/ST-999 | 404 |
| … als XML | curl.exe -i -H "Accept: application/xml" http://localhost:8080/api/v1/stations/ST-999 | 404 als <error> |

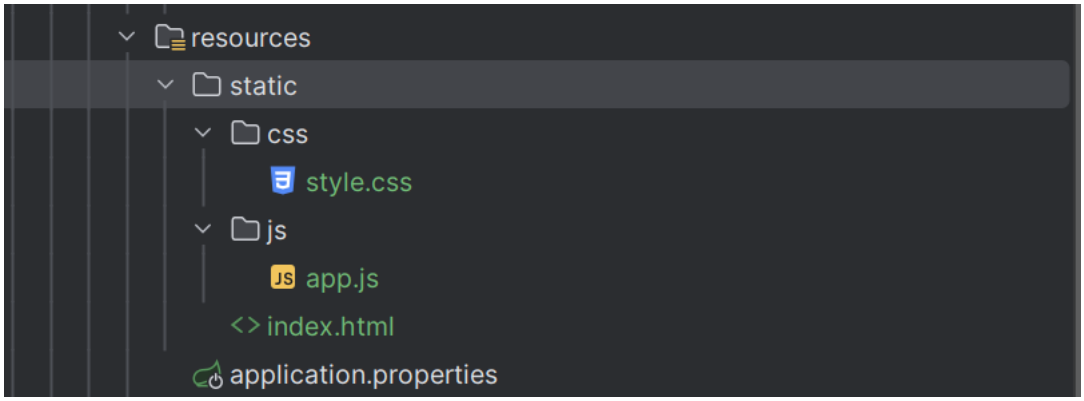
| Test | Befehl | Erwartet |
|---|---|---|
| from nach to | curl.exe -i "http://localhost:8080/api/v1/stations/ST-001/measurements?from=2026-05-01T12:00:00Z&to=2026-05-01T08:00:00Z" | 400 |
| Ungültiges Datum | curl.exe -i "http://localhost:8080/api/v1/stations/ST-001/measurements?from=gestern" | 400 |
| Ungültige Warnstufe | curl.exe -i "http://localhost:8080/api/v1/stations?warningLevel=FOO" | 400 |
| Negatives limit | curl.exe -i "http://localhost:8080/api/v1/stations/ST-001/measurements?limit=-1" | 400 |
| Zu großes limit | curl.exe -i "http://localhost:8080/api/v1/stations/ST-001/measurements?limit=5000" | 400 |
| Falsches Format | curl.exe -i -H "Accept: text/plain" http://localhost:8080/api/v1/stations/ST-001 | 406 mit Meldung |

### 6.4 Commit

## 7. Webclient

### 7.1 Ordner anlegen

Am ende soll es so aussehen :




### 7.2 static/index.html

### 7.3 static/css/style.css

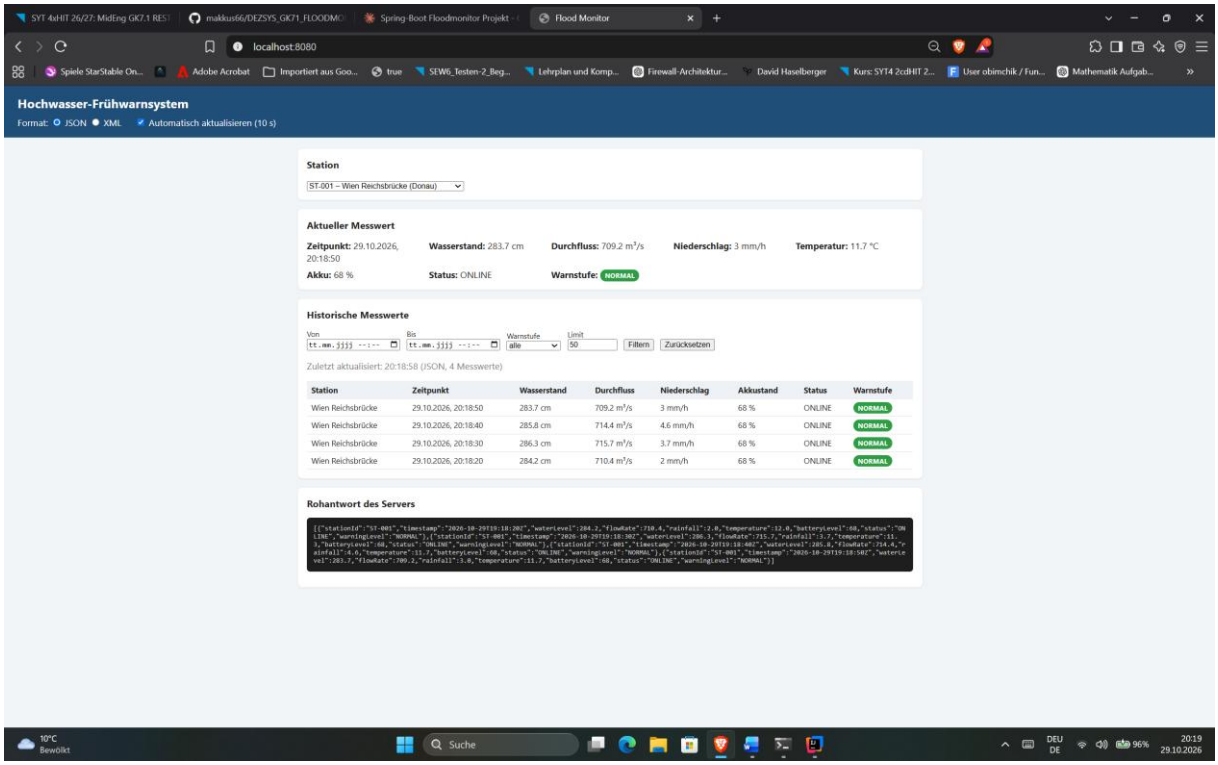
### 7.4 static/js/app.js

### 7.5 Testen

Sever starten . Dann auf http://localhost:8080/ gehen

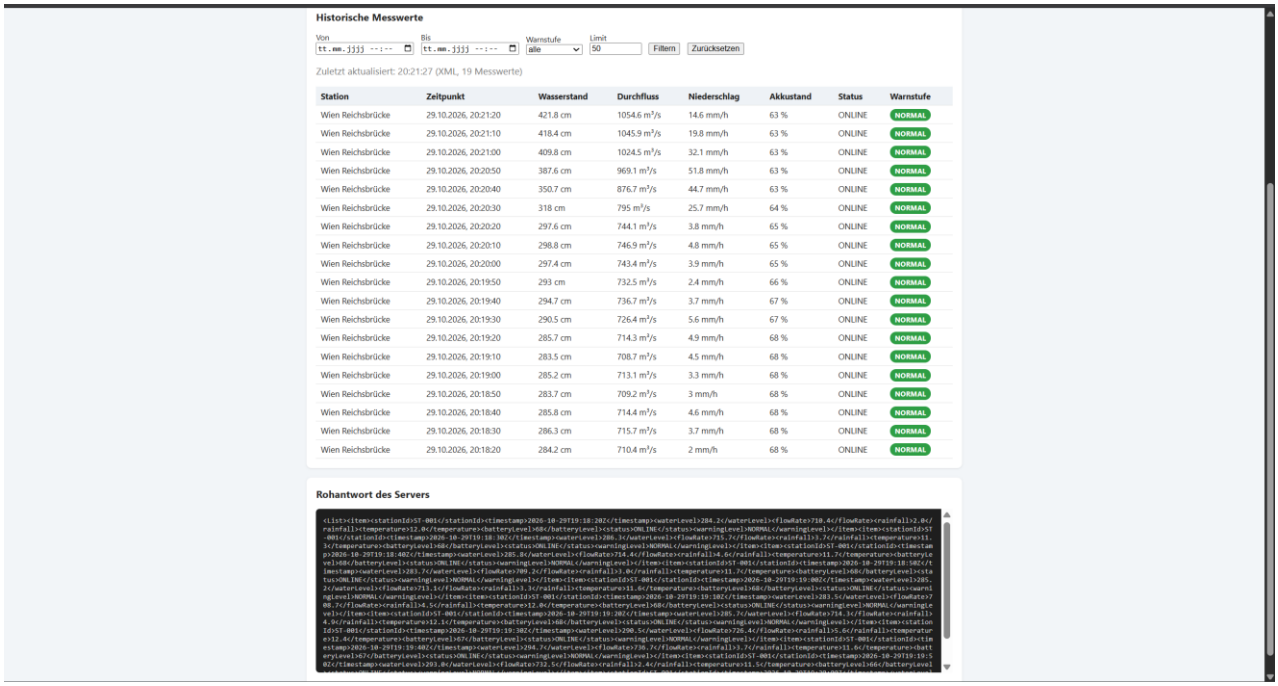
## Gesamtansicht (JSON)

Mein Webclient lädt die Stationen und zeigt den aktuellen Messwert sowie die historischen Messwerte in einer Tabelle an. Die Warnstufen sind farbig markiert.



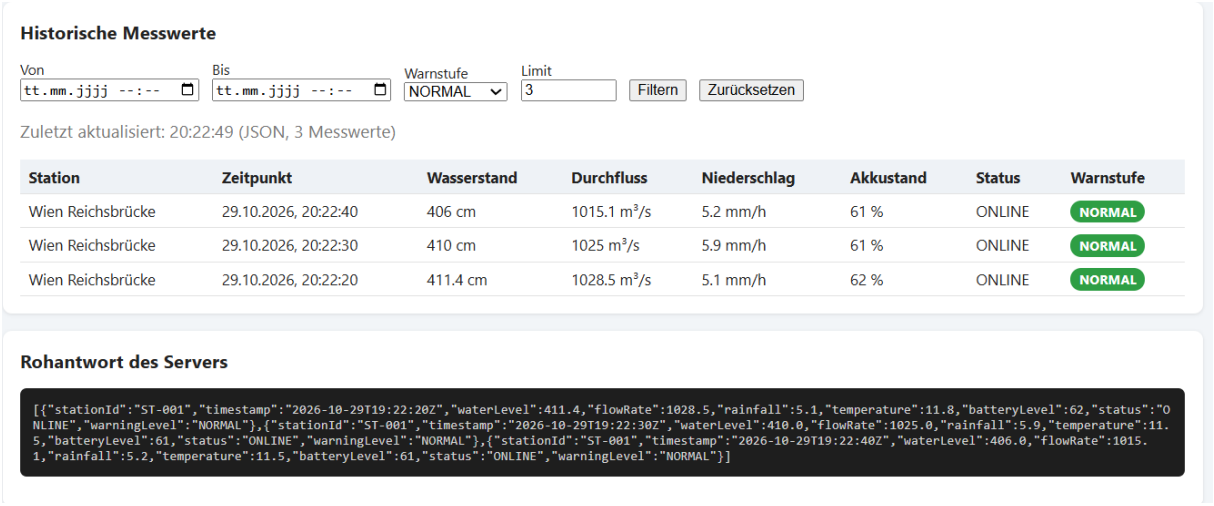

## XML-Modus

Ich habe auf XML umgeschaltet. Der Client sendet jetzt Accept: application/xml, wertet die XML-Antwort aus und zeigt dieselben Informationen an wie bei JSON.



## Filter nach Warnstufe und Limit

Ich habe nach der Warnstufe NORMAL gefiltert und die Anzahl auf 3 Messwerte begrenzt.



**Historische Messwerte**

Von: tt.mm.jjjj --:-- | Bis: tt.mm.jjjj --:-- | Warnstufe: NORMAL | Limit: 3 | Filtern | Zurücksetzen

Zuletzt aktualisiert: 20:22:49 (JSON, 3 Messwerte)

| Station | Zeitpunkt | Wasserstand | Durchfluss | Niederschlag | Akkustand | Status | Warnstufe |
|---|---|---|---|---|---|---|---|
| Wien Reichsbrücke | 29.10.2026, 20:22:40 | 406 cm | 1015.1 m³/s | 5.2 mm/h | 61 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:22:30 | 410 cm | 1025 m³/s | 5.9 mm/h | 61 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:22:20 | 411.4 cm | 1028.5 m³/s | 5.1 mm/h | 62 % | ONLINE | NORMAL |

**Rohantwort des Servers**


```
[{"stationId":"ST-001","timestamp":"2026-10-29T19:22:20Z","waterLevel":411.4,"flowRate":1028.5,"rainfall":5.1,"temperature":11.8,"batteryLevel":62,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:22:30Z","waterLevel":410.0,"flowRate":1025.0,"rainfall":5.9,"temperature":11.5,"batteryLevel":61,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:22:40Z","waterLevel":406.0,"flowRate":1015.1,"rainfall":5.2,"temperature":11.5,"batteryLevel":61,"status":"ONLINE","warningLevel":"NORMAL"}]
```


## Filter nach Zeitraum

Ich habe die Messwerte auf einen bestimmten Zeitraum eingeschränkt. Es werden nur Messwerte zwischen Von und Bis angezeigt.



**Historische Messwerte**

Von: 29.10.2026 20:15 | Bis: 29.10.2026 20:23 | Warnstufe: alle | Limit: 80 | Filtern | Zurücksetzen

Zuletzt aktualisiert: 20:23:57 (JSON, 29 Messwerte)

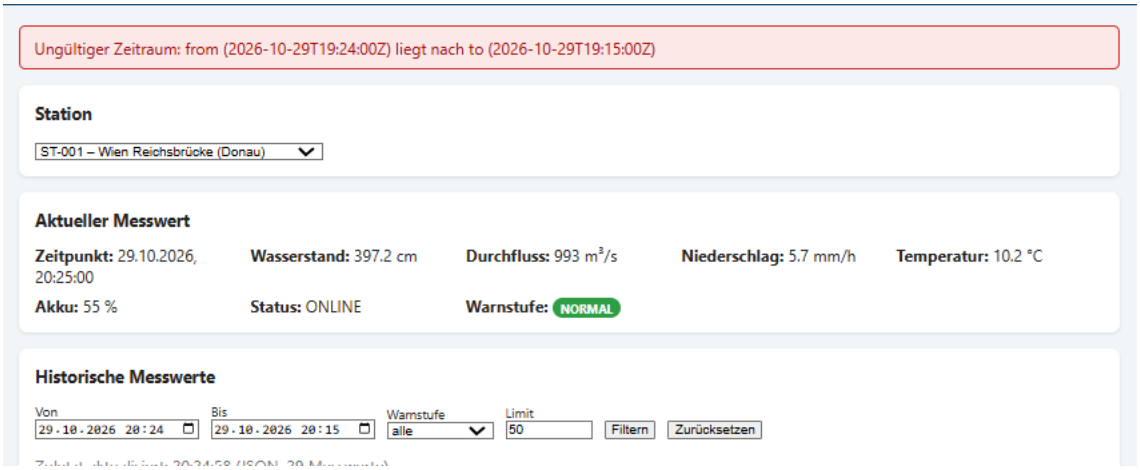
| Station | Zeitpunkt | Wasserstand | Durchfluss | Niederschlag | Akkustand | Status | Warnstufe |
|---|---|---|---|---|---|---|---|
| Wien Reichsbrücke | 29.10.2026, 20:23:00 | 405.8 cm | 1014.5 m³/s | 5.6 mm/h | 60 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:22:50 | 405.9 cm | 1014.9 m³/s | 4.8 mm/h | 60 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:22:40 | 406 cm | 1015.1 m³/s | 5.2 mm/h | 61 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:22:30 | 410 cm | 1025 m³/s | 5.9 mm/h | 61 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:22:20 | 411.4 cm | 1028.5 m³/s | 5.1 mm/h | 62 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:22:10 | 414.6 cm | 1036.6 m³/s | 3.7 mm/h | 62 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:22:00 | 416.4 cm | 1040.9 m³/s | 4.5 mm/h | 62 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:21:50 | 421.5 cm | 1053.7 m³/s | 6.9 mm/h | 62 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:21:40 | 422.9 cm | 1057.2 m³/s | 7.5 mm/h | 62 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:21:30 | 424.3 cm | 1060.6 m³/s | 9.7 mm/h | 63 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:21:20 | 421.8 cm | 1054.6 m³/s | 14.6 mm/h | 63 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:21:10 | 418.4 cm | 1045.9 m³/s | 19.8 mm/h | 63 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:21:00 | 409.8 cm | 1024.5 m³/s | 32.1 mm/h | 63 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:20:50 | 387.6 cm | 969.1 m³/s | 51.8 mm/h | 63 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:20:40 | 350.7 cm | 876.7 m³/s | 44.7 mm/h | 63 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:20:30 | 318 cm | 795 m³/s | 25.7 mm/h | 64 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:20:20 | 297.6 cm | 744.1 m³/s | 3.8 mm/h | 65 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:20:10 | 298.8 cm | 746.9 m³/s | 4.8 mm/h | 65 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:20:00 | 297.4 cm | 743.4 m³/s | 3.9 mm/h | 65 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:19:50 | 293 cm | 732.5 m³/s | 2.4 mm/h | 66 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:19:40 | 294.7 cm | 736.7 m³/s | 3.7 mm/h | 67 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:19:30 | 290.5 cm | 726.4 m³/s | 5.6 mm/h | 67 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:19:20 | 285.7 cm | 714.3 m³/s | 4.9 mm/h | 68 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:19:10 | 283.5 cm | 708.7 m³/s | 4.5 mm/h | 68 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:19:00 | 285.2 cm | 713.1 m³/s | 3.3 mm/h | 68 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:18:50 | 283.7 cm | 709.2 m³/s | 3 mm/h | 68 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:18:40 | 285.8 cm | 714.4 m³/s | 4.6 mm/h | 68 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:18:30 | 286.3 cm | 715.7 m³/s | 3.7 mm/h | 68 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:18:20 | 284.2 cm | 710.4 m³/s | 2 mm/h | 68 % | ONLINE | NORMAL |

**Rohantwort des Servers**


```
[{"stationId":"ST-001","timestamp":"2026-10-29T19:18:20Z","waterLevel":284.2,"flowRate":710.4,"rainfall":2.0,"temperature":12.0,"batteryLevel":68,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:18:30Z","waterLevel":286.3,"flowRate":715.7,"rainfall":3.7,"temperature":11.9,"batteryLevel":68,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:18:40Z","waterLevel":285.8,"flowRate":714.4,"rainfall":4.6,"temperature":12.2,"batteryLevel":68,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:18:50Z","waterLevel":283.7,"flowRate":709.2,"rainfall":3.0,"temperature":11.7,"batteryLevel":68,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:19:00Z","waterLevel":285.2,"flowRate":713.1,"rainfall":3.3,"temperature":11.8,"batteryLevel":68,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:19:10Z","waterLevel":283.5,"flowRate":708.7,"rainfall":4.5,"temperature":12.0,"batteryLevel":68,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:19:20Z","waterLevel":285.7,"flowRate":714.3,"rainfall":4.9,"temperature":12.1,"batteryLevel":68,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:19:30Z","waterLevel":290.5,"flowRate":726.4,"rainfall":5.6,"temperature":12.0,"batteryLevel":67,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:19:40Z","waterLevel":294.7,"flowRate":736.7,"rainfall":3.7,"temperature":11.6,"batteryLevel":67,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:19:50Z","waterLevel":293,"flowRate":732.5,"rainfall":2.4,"temperature":11.5,"batteryLevel":66,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:20:00Z","waterLevel":297.4,"flowRate":743.4,"rainfall":3.9,"temperature":11.9,"batteryLevel":65,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:20:10Z","waterLevel":298.8,"flowRate":746.9,"rainfall":4.8,"temperature":11.3,"batteryLevel":65,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:20:20Z","waterLevel":297.6,"flowRate":744.1,"rainfall":3.8,"temperature":11.6,"batteryLevel":65,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:20:30Z","waterLevel":318,"flowRate":795.0,"rainfall":25.7,"temperature":11.6,"batteryLevel":64,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:20:40Z","waterLevel":350.7,"flowRate":876.7,"rainfall":44.7,"temperature":11.2,"batteryLevel":63,"status":"ONLINE","warningLevel":"NORMAL"},{"stationId":"ST-001","timestamp":"2026-10-29T19:20:50Z","waterLevel":387.
```


## Fehlermeldung bei ungültigem Zeitraum

Wenn der Startzeitpunkt nach dem Endzeitpunkt liegt, antwortet der Server mit 400 Bad Request. Der Client zeigt die Fehlermeldung verständlich an.




## Automatische Aktualisierung

Der Client aktualisiert die Daten alle 10 Sekunden automatisch. Im zweiten Bild ist die Uhrzeit neuer und ein neuer Messwert ist dazugekommen.

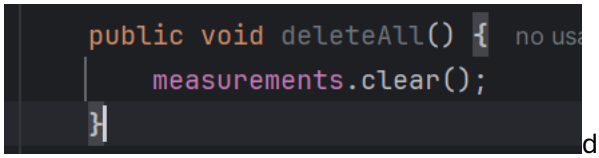
| Station | Zeitpunkt | Wasserstand | Durchfluss | Niederschlag | Akkustand | Status | Warnstufe |
|---|---|---|---|---|---|---|---|
| Wien Reichsbrücke | 29.10.2026, 20:25:50 | 417.3 cm | 1043.4 m³/s | 19.3 mm/h | 54 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:25:40 | 407.1 cm | 1017.8 m³/s | 27.5 mm/h | 54 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:25:30 | 388.8 cm | 971.9 m³/s | 2.9 mm/h | 54 % | ONLINE | NORMAL |
| Wien Reichsbrücke | 29.10.2026, 20:25:20 | 392.4 cm | 980.9 m³/s | 3 mm/h | 54 % | ONLINE | NORMAL |

# 8. Architektur

| Schicht | Paket / Ordner | Inhalt |
|---|---|---|
| Model | model | Station, Location, Measurement, Alert, Statistics, WarningLevel, StationStatus |
| Repository | repository | StationRepository, MeasurementRepository (speichern im Arbeitsspeicher) |
| Service | service | StationService, MeasurementService, WarningLevelService, SimulationService |
| Controller | controller | StationController, AlertController |
| Exception Handling | exception | GlobalExceptionHandler, ErrorResponse, NotFoundException, BadRequestException |
| Client | resources/ static | index.html, style.css, app.js |

# Punkt 9: Automatisierte Tests

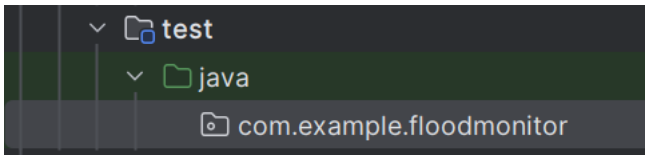
9.1 Eine Methode in MeasurementRepository.java ergänzen



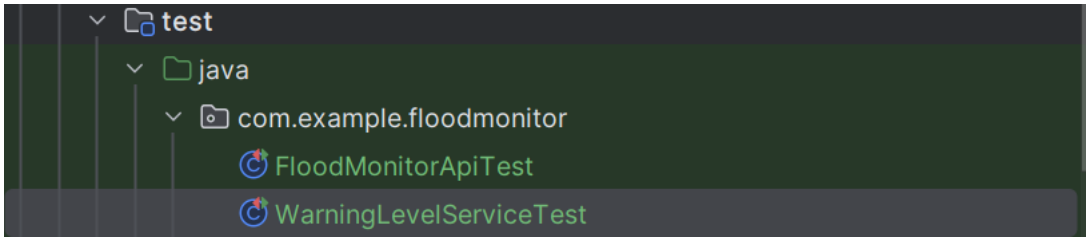
dies hinzugefügt am ende

9.2 Test-Ordner anlegen

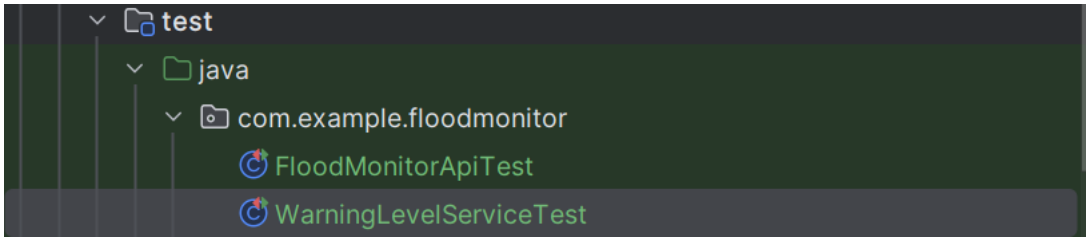
Rechtsklick auf src → New → Directory → **test/java** auswählen (IntelliJ schlägt das vor).
Darin: Rechtsklick auf java → New → Package → com.example.floodmonitor




## 9.3 FloodMonitorApiTest.java




## 9.4 WarningLevelServiceTest.java




## 9.5 Tests ausführen

Ausführen mit .\gradlew.bat test

Ich habe alle automatisierten Tests mit Gradle ausgeführt, alle sind erfolgreich.

```
BUILD SUCCESSFUL in 9s
```

Für die Tests deaktiviere ich die Simulation mit simulation.enabled=false und lege vor jedem Test feste Messwerte an. Dadurch sind die Tests nicht vom Zufall abhängig und schlagen nie zufällig fehl. Die Warnstufen teste ich direkt am WarningLevelService.

| **Test aus der Angabe** | **Testmethode** |
|---|---|
| Alle Stationen als JSON | allStationsAsJson |

| Test aus der Angabe | Testmethode |
|---|---|
| Alle Stationen als XML | allStationsAsXml |
| Existierende Station | existingStation |
| Unbekannte Station | unknownStation |
| Aktuellster Messwert | latestMeasurement |
| Filterung nach Zeitraum | filterByTimeRange |
| Ungültiger Zeitraum | invalidTimeRange |
| Warnstufe NORMAL / WARNING / CRITICAL | normalBelowWarningLevel, warningFromWarningLevel, criticalFromCriticalLevel |
| Nicht unterstütztes Format | unsupportedFormat |
| Maximales Limit | maxLimit |

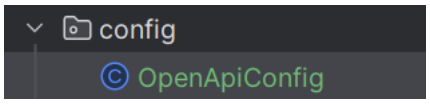
# Punkt 10: API-Dokumentation mit OpenAPI

10.1 Dependency in build.gradle

Ich habe springdoc-openapi eingebunden. Damit wird die OpenAPI-Dokumentation automatisch aus dem Code erzeugt.

```
dependencies { Add Starters...
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'com.fasterxml.jackson.dataformat:jackson-dataformat-xml'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.3.0'
}
```

## 10.2 Neues Paket config → OpenApiConfig.java




## 10.3 StationController.java komplett ersetzen

Gleiche Logik wie vorher, aber mit Beschreibungen für die Doku

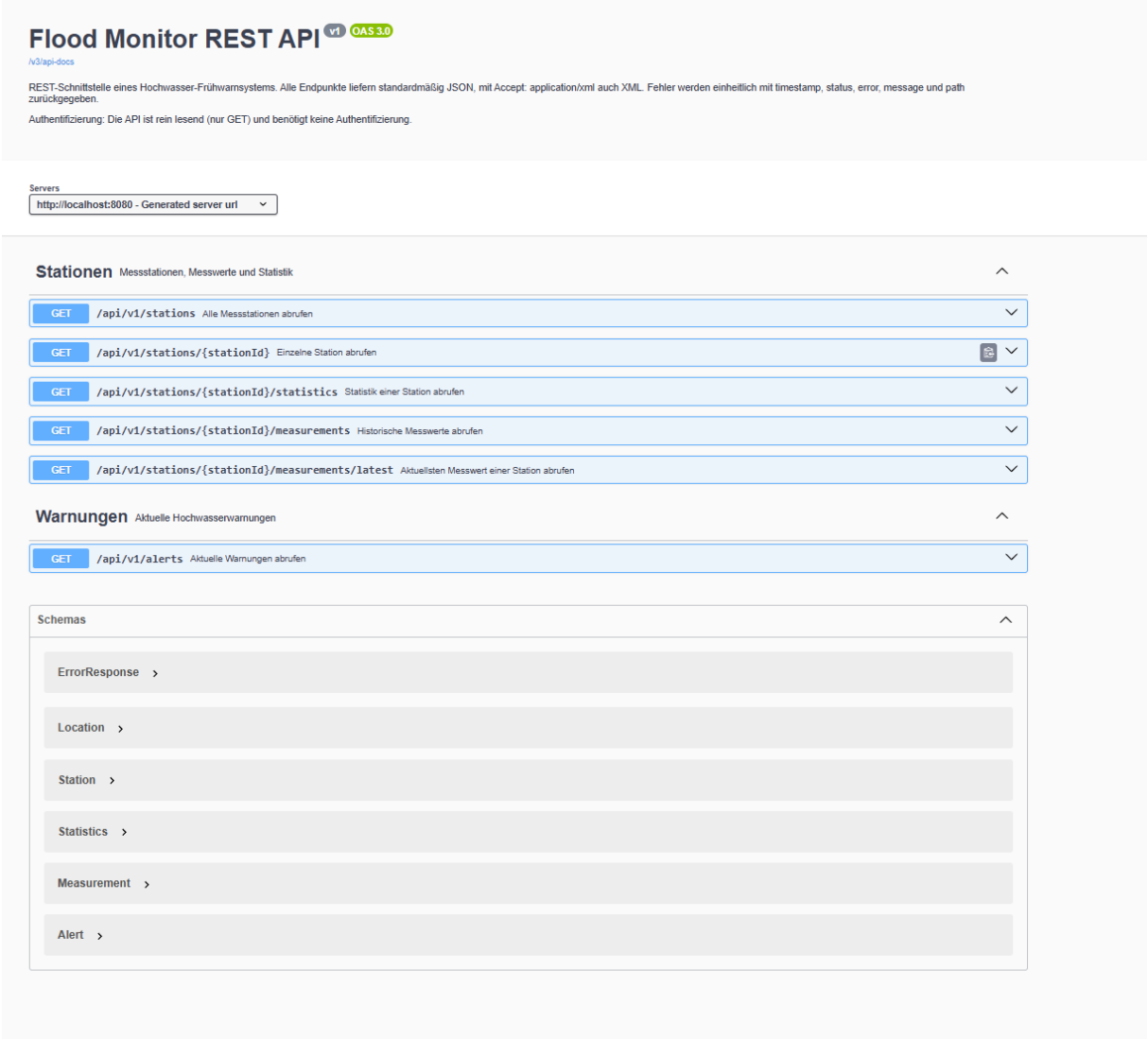
## 10.4 AlertController.java komplett ersetzen

Gleiche Logik wie vorher, aber mit Beschreibungen für die Doku

## 10.5 Testen

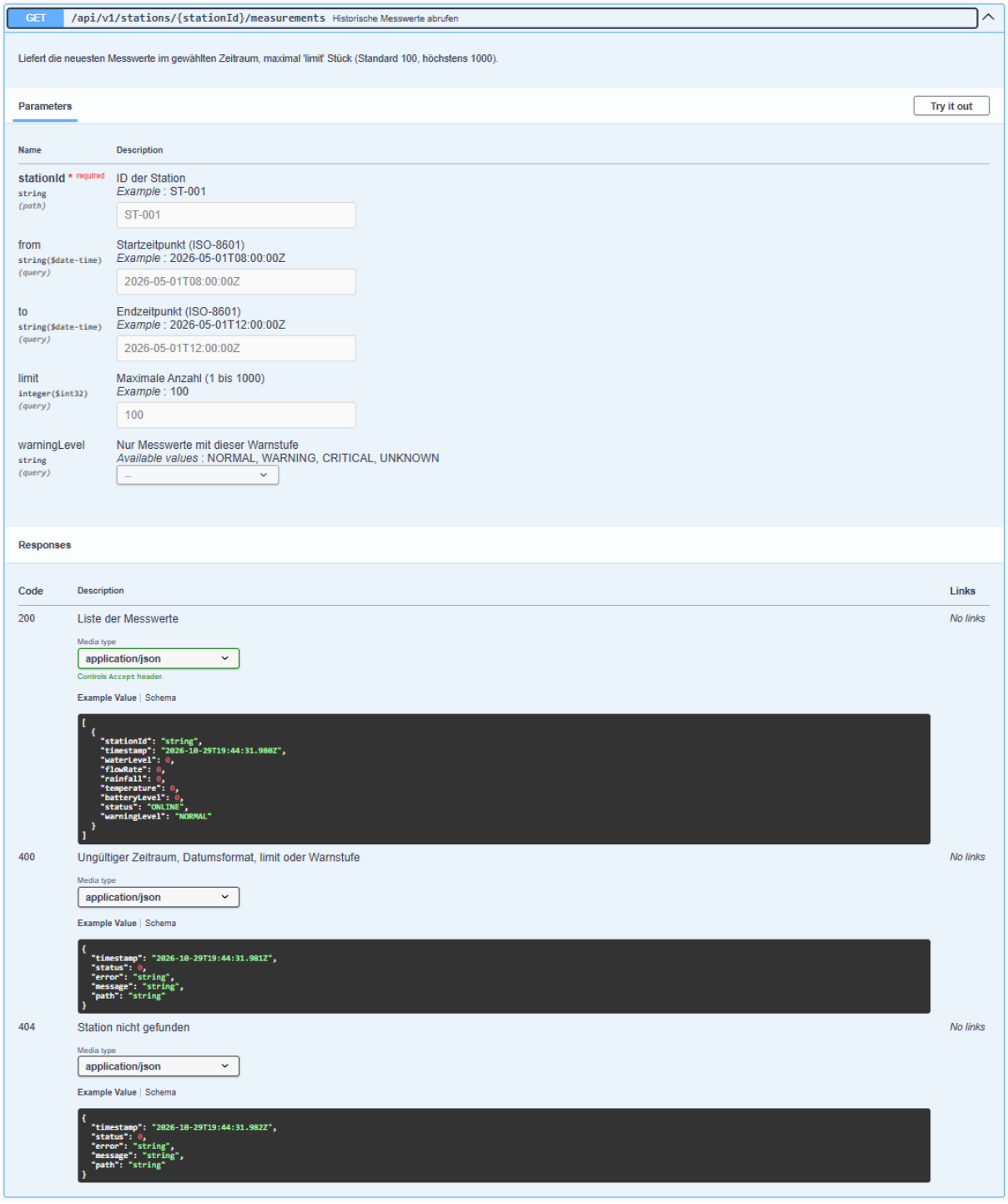
Server starten und dann auf diese Seite gehen : http://localhost:8080/swagger-ui.html

Die API-Dokumentation ist über Swagger UI unter /swagger-ui.html erreichbar. Sie zeigt alle Endpunkte mit Methode, Pfad und Beschreibung sowie den Hinweis zur Authentifizierung.



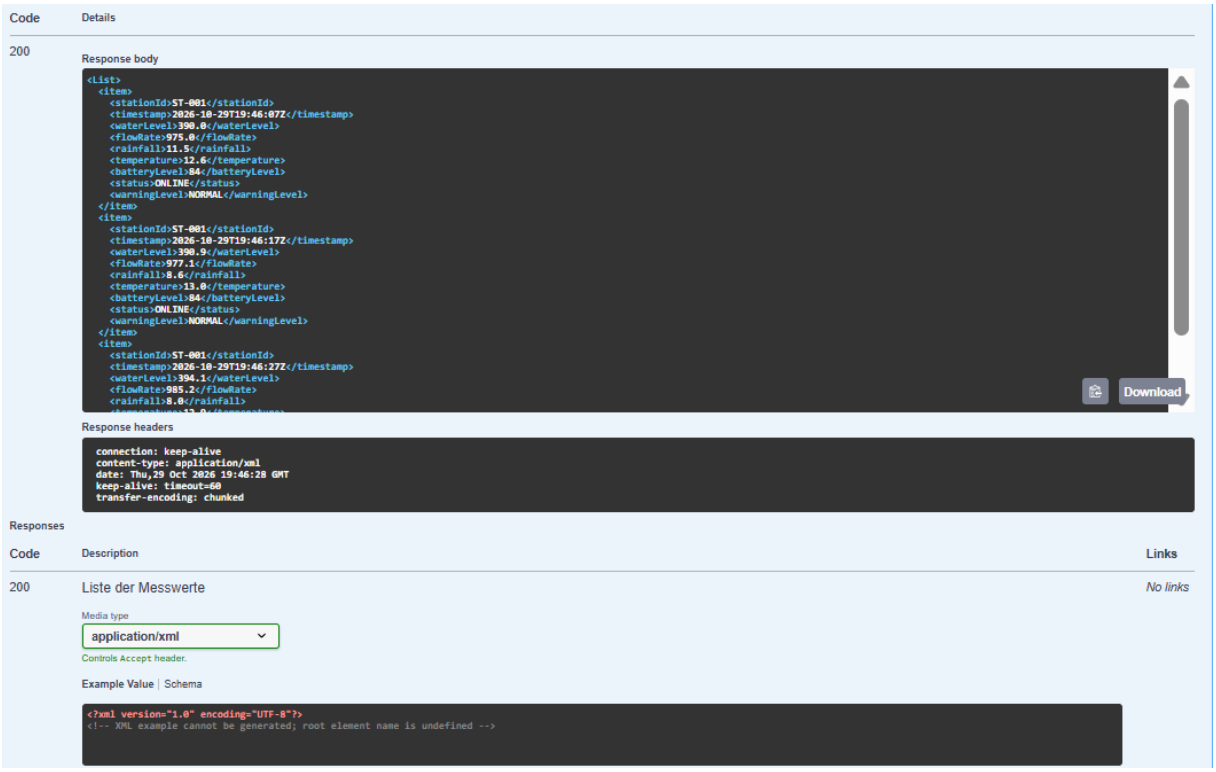

## Endpunkt mit Parametern und Statuscodes

*Für jeden Endpunkt sind die Parameter mit Beschreibung und Beispielwert dokumentiert.“*
*„Außerdem sind alle möglichen Statuscodes und das Fehlerformat dokumentiert.*



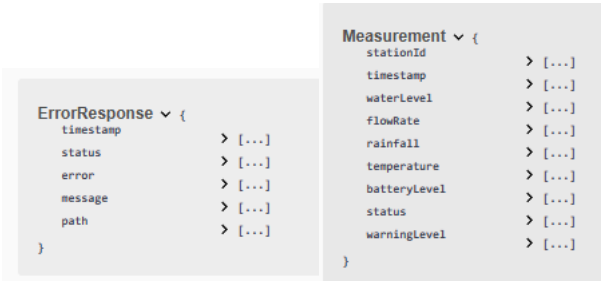

## Direkt in Swagger testen (XML)

Ich habe den Endpunkt direkt in Swagger UI getestet und als Format XML gewählt. Der Server antwortet mit 200 und liefert die Messwerte als XML.




## Schemas

Unter Schemas sind die Datenstrukturen der Antworten beschrieben, darunter Measurement und das einheitliche Fehlerformat ErrorResponse.

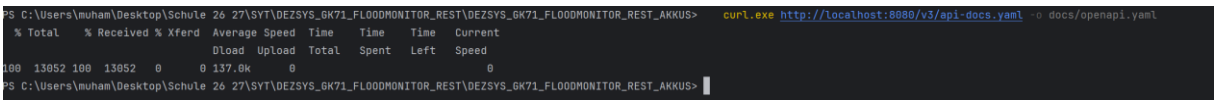


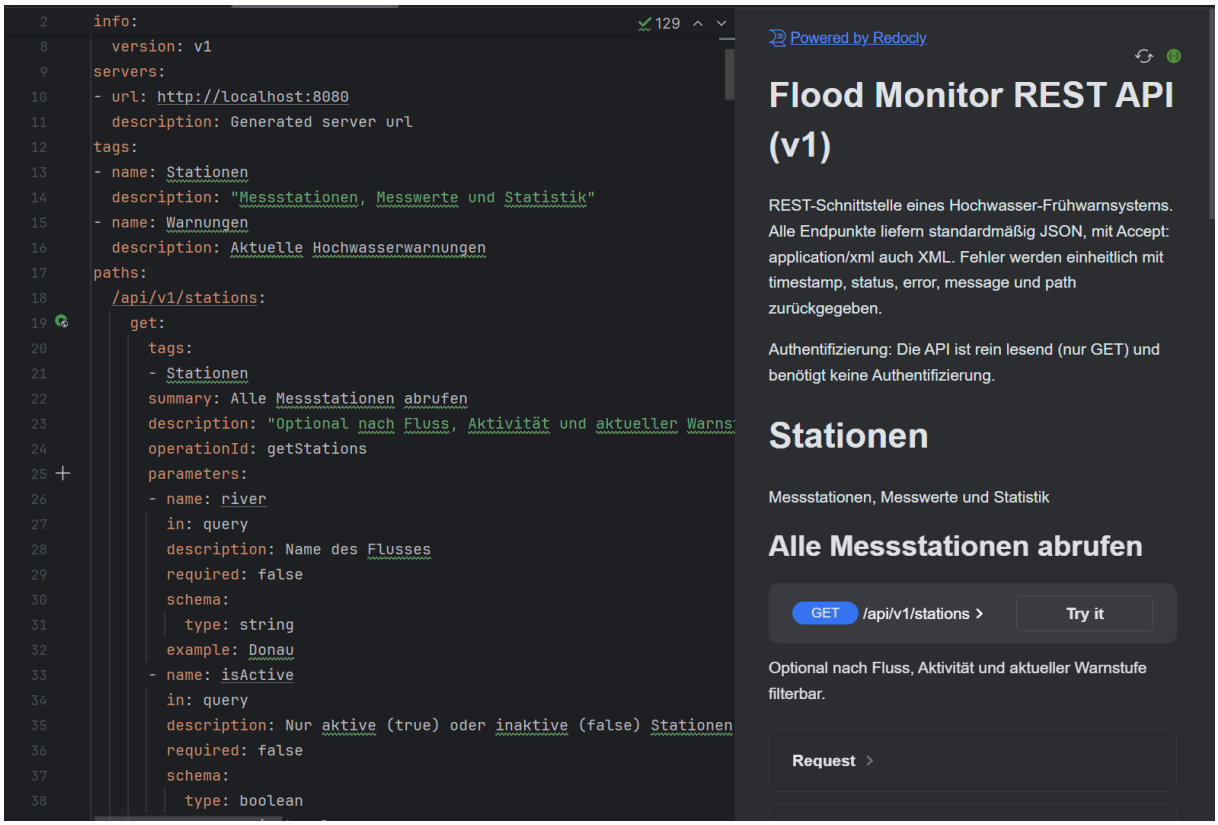




## openapi.yaml

Im zweiten terminal dann dies eingeben :






Ich habe die OpenAPI-Spezifikation als openapi.yaml im Ordner docs gespeichert, damit sie auch ohne laufenden Server im Repository liegt.

# 12. Fragestellungen

## 1. Was sind die Hauptmerkmale von JSON?

JSON (JavaScript Object Notation) ist ein leichtgewichtiges, textbasiertes Datenformat. Es kennt zwei Strukturen: Objekte (ungeordnete Schlüssel-Wert-Paare in { }) und Arrays (geordnete Listen in [ ]). Als Werte sind Strings, Zahlen, true, false, null, Objekte und Arrays erlaubt. JSON ist für Menschen gut lesbar, sprachunabhängig und mit wenig Overhead verbunden, weil es keine schließenden Tags gibt. Es gibt keine Kommentare und keine eingebauten Datentypen für z. B. Datum, deshalb übertrage ich Zeitpunkte als ISO-8601-String. JSON ist heute das Standardformat für Web-APIs.

*Quellen:* https://www.json.org/json-de.html, RFC 8259: https://datatracker.ietf.org/doc/html/rfc8259

## 2. Was sind die Hauptmerkmale von XML?

XML (Extensible Markup Language) ist eine textbasierte Auszeichnungssprache mit frei definierbaren Elementen in Start- und End-Tags (z. B. <waterLevel>318.4</waterLevel>). Ein XML-Dokument hat genau ein Wurzelelement und muss „wohlgeformt“ sein (korrekt verschachtelt und geschlossen). XML unterstützt Attribute, Namespaces und Kommentare und kann mit XML Schema (XSD) oder DTD auf Gültigkeit geprüft werden. Es gibt viele Werkzeuge wie

XPath oder XSLT. Nachteil gegenüber JSON: XML ist durch die Tags deutlich länger und aufwendiger zu verarbeiten.

*Quellen:* W3C XML 1.0: https://www.w3.org/TR/xml/

## 3. Wie unterscheidet die Applikation, ob JSON oder XML zurückgegeben wird?

Über **Content Negotiation** mit dem HTTP-Header Accept. Der Client schickt z. B. Accept: application/xml. Spring MVC vergleicht diesen Wert mit den Formaten, die die vorhandenen HttpMessageConverter erzeugen können. Für JSON ist das der Jackson-JSON-Converter, für XML der Jackson-XML-Converter, der durch die Abhängigkeit jackson-dataformat-xml dazukommt. In meinen Controllern sind beide Formate zusätzlich mit produces angegeben. Ohne Accept-Header wird JSON geliefert, weil Spring Boot den XML-Converter ans Ende der Liste stellt. Passt kein Converter, antwortet Spring mit **406 Not Acceptable**.

*Quellen:* Spring Framework Reference, Content Negotiation: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-config/content-negotiation.html, RFC 9110 Abschnitt 12.5.1 (Accept): https://datatracker.ietf.org/doc/html/rfc9110#section-12.5.1

## 4. Was zeichnet ein REST-Interface aus und warum wird es so oft verwendet?

REST (Representational State Transfer) ist ein Architekturstil, den Roy Fielding 2000 beschrieben hat. Die wichtigsten Prinzipien:

- **Ressourcen** werden über eindeutige URLs angesprochen (z. B. /api/v1/stations/ST-001).
- **Einheitliche Schnittstelle:** Die HTTP-Methoden (GET, POST, PUT, PATCH, DELETE) haben eine feste Bedeutung, Ergebnisse werden über Statuscodes mitgeteilt.
- **Zustandslosigkeit:** Jede Anfrage enthält alle nötigen Informationen, der Server speichert keinen Sitzungszustand.
- **Client-Server-Trennung** und **Cachebarkeit** von Antworten.
- **Repräsentationen:** Dieselbe Ressource kann in verschiedenen Formaten (JSON, XML) geliefert werden.

REST ist so verbreitet, weil es direkt auf HTTP aufbaut. Jede Programmiersprache und jeder Browser kann es ohne zusätzliche Bibliotheken nutzen, es ist einfach zu verstehen und zu testen (Browser, curl), es skaliert gut wegen der Zustandslosigkeit, und es ist gut durch Proxys und Caches unterstützt.

*Quellen:* Fielding, Dissertation Kapitel 5:
https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

## 5. Unterschied zwischen GET, POST, PUT und PATCH?

| Methode | Zweck | Sicher | Idempotent |
|---|---|---|---|
| GET | Ressource lesen, keine Änderung am Server | ja | ja |
| POST | Neue Ressource anlegen oder Verarbeitung auslösen | nein | nein |
| PUT | Ressource vollständig ersetzen (oder unter bekannter URL anlegen) | nein | ja |
| PATCH | Ressource teilweise ändern | nein | nicht garantiert |

„Sicher“ heißt, der Server-Zustand wird nicht verändert. „Idempotent“ heißt, mehrfaches Ausführen hat dieselbe Wirkung wie einmaliges. Beispiel: Zweimal PUT mit denselben Daten ergibt dasselbe Ergebnis, zweimal POST legt zwei Stationen an.

*Quellen:* RFC 9110 Abschnitt 9 (Methoden): https://datatracker.ietf.org/doc/html/rfc9110#section-9, RFC 5789 (PATCH): https://datatracker.ietf.org/doc/html/rfc5789

## 6. Welche Stellen im Code muss ich für eine neue Methode / einen neuen Pfad ändern?

In meinem Projekt:

1. **Controller** (StationController oder neuer Controller): neue Methode mit passender Annotation (@GetMapping, @PostMapping, ...) und Pfad, Parameter mit @PathVariable, @RequestParam oder @RequestBody.
2. **Service** (z. B. MeasurementService): die eigentliche Logik, weil Controller keine Geschäftslogik enthalten sollen.
3. **Repository**, falls neue Daten gelesen oder gespeichert werden.
4. **Model**, falls eine neue Datenklasse gebraucht wird (z. B. wie Statistics).
5. **GlobalExceptionHandler**, falls neue Fehlerfälle dazukommen.
6. **OpenAPI-Annotationen** (@Operation, @ApiResponse), damit der Endpunkt dokumentiert ist.
7. **Tests** für den neuen Endpunkt.
8. Den **Webclient** (app.js), falls er den Endpunkt nutzen soll.

*Quellen:* Spring Guide „Building a RESTful Web Service“: https://spring.io/guides/gs/rest-service/, Spring Reference „Annotated Controllers“: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller.html

## 7. Kann man OpenAPI und Swagger vergleichen?

Nur bedingt, weil sie nicht dasselbe sind:

- **OpenAPI** ist eine **Spezifikation**, also ein herstellerunabhängiger Standard, wie man REST-APIs in JSON oder YAML beschreibt (Endpunkte, Parameter, Antworten, Schemas, Authentifizierung). Sie wird von der OpenAPI Initiative unter der Linux Foundation weiterentwickelt.
- **Swagger** ist der **ursprüngliche Name** dieser Spezifikation und heute der Name einer **Sammlung von Werkzeugen** der Firma SmartBear, die mit OpenAPI arbeiten: Swagger UI (interaktive Doku), Swagger Editor, Swagger Codegen.

2015 wurde die Swagger Specification an die OpenAPI Initiative übergeben und in OpenAPI umbenannt (ab Version 3.0). Kurz gesagt: OpenAPI ist das „Was“ (das Format), Swagger das „Womit“ (die Tools). In meinem Projekt erzeugt springdoc die OpenAPI-Beschreibung (/v3/api-docs), und **Swagger UI** stellt sie unter /swagger-ui.html dar.

*Quellen:* https://swagger.io/docs/specification/about/, https://www.openapis.org/about

## 8. Wo liegt im Code-Snippet der Fehler?

```
@GetMapping("/stations/add")
public String createStation(@RequestParam String name, @RequestParam double warning) {
  stationService.save(new Station(name, warning));
  return "Station successfully created!";
}
```

Der Hauptfehler: Es wird **GET zum Anlegen einer Ressource** verwendet. GET muss laut HTTP **sicher** sein, darf also den Server-Zustand nicht ändern. Hier wird aber bei jedem Aufruf eine neue Station gespeichert. Das ist gefährlich, weil Browser, Suchmaschinen-Crawler oder Prefetching GET-Anfragen automatisch ausführen oder wiederholen und Antworten gecacht werden können. So könnten ungewollt Stationen angelegt werden.

Weitere Probleme:

- Das **Verb „add“ im Pfad** widerspricht REST. Die Aktion ergibt sich aus der HTTP-Methode, der Pfad benennt nur die Ressource.
- Die Daten kommen als **Query-Parameter** statt als **Request-Body**.
- Es wird **ein Text-String** zurückgegeben statt der angelegten Ressource, und der Statuscode ist 200 statt **201 Created**.

So wäre es richtig:

```
@PostMapping("/stations")
public ResponseEntity<Station> createStation(@RequestBody Station station) {
  Station saved = stationService.save(station);
  return ResponseEntity.created(URI.create("/api/v1/stations/" + saved.getId())).body(saved);
}
```

*Quellen:* RFC 9110 Abschnitt 9.2.1 (sichere Methoden): https://datatracker.ietf.org/doc/html/rfc9110#section-9.2.1, Abschnitt 15.3.2 (201 Created): https://datatracker.ietf.org/doc/html/rfc9110#section-15.3.2

# AI-Prompts

Für diese Aufgabe habe ich Claude (Anthropic) verwendet. Die wichtigsten Prompts waren:

| Nr. | Prompt | Verwendung |
|---|---|---|
| 1 | Helfe mir bitte beim erstellen des Datenmodells (Station, Location, Measurement, Enums) | Punkt 2 |
| 2 | Wie berechne ich die Warnstufe serverseitig anhand der Grenzwerte? | Punkt 3 |
| 3 | Wie Implementiere ich die REST-Endpunkte aus der Angabe mit Filtern, Statistik und Warnungen. | Punkt 4 |
| 4 | Wie teste ich, ob JSON und XML über den Accept-Header funktionieren? | Punkt 5 |
| 5 | Was muss ich genau beim erstellen eines Webclient mit HTML, CSS und jQuery, der JSON und XML machen | Punkt 7 |
| 6 | Helfe mir bitte bei den automatisierten Tests. Was muss ich genaz machen ? | Punkt 9 |
| 7 | Wie dokumentiere ich die API mit OpenAPI und Swagger UI? | Punkt 10 |